

I ported speculative decoding to a robot policy

Aarav Raina · May 15, 2026 · 12 min read

I just got home from a hackathon. The brief was simple: build a marimo notebook with Cursor that does something cool with one of a handful of public datasets. PyData and Cursor put it on at Moderna HQ. Three hours. One dataset. One insight. Show up, ship, leave.

I picked the robotics dataset because I thought robotics was the coolest direction, and because most other people would pick MBTA or NASA exoplanets and I wanted a different lane.

This post is about what I ended up building, what I got wrong along the way, and the inference trick I lifted from LLM serving research to make a 263M-parameter diffusion policy hit a 10 Hz control deadline on a laptop CPU.

The dataset

Hugging Face's [LeRobot](#) team maintains a collection of pretrained robot policies and datasets. The canonical small one is `lerobot/pusht`: a single arm pushing a T-shaped block across a 2D plane, recorded at 10 Hz. ~200 episodes, ~25,000 frames. Each frame is a 96x96 RGB image plus a 2D state vector (where the pusher is) and a 2D action (where the pusher is going next). Tiny but real.

The pretrained policy for this dataset is `lerobot/diffusion_pusht`. It's a diffusion-based imitation policy. Given an observation, it runs an iterative denoising loop (100 steps by default) and outputs an action. 263 million parameters. The implementation is a U-Net, structurally similar to the U-Net in image diffusion models but operating in action-space.

First thing I did was load it on CPU and time a single forward pass.

```
1.2 seconds per call.
```

Pusht's control loop runs at 10 Hz. That's a 100 ms budget per frame. I was missing the deadline by roughly 12x.

The standard fix is to turn the knob down

Diffusion policies have one obvious knob: the number of denoising steps. The default is 100. The community standard for fast inference is to lower it to something like 20 or 10. I swept it:

denoising steps	single-call latency	action MSE on sample
50	~2.6 s	234
20	~1.0 s	263
10	~550 ms	235
5	~270 ms	193

LeRobot's chunking helps. The policy plans a chunk of `n_action_steps=8` actions per forward pass, then serves them open-loop. So if a forward pass takes 1.24 s at 20 steps, the effective per-frame cost is $1.24 / 8 = 154$ ms. Still over the budget. At 5 steps, it drops to 34 ms per frame: under the deadline.

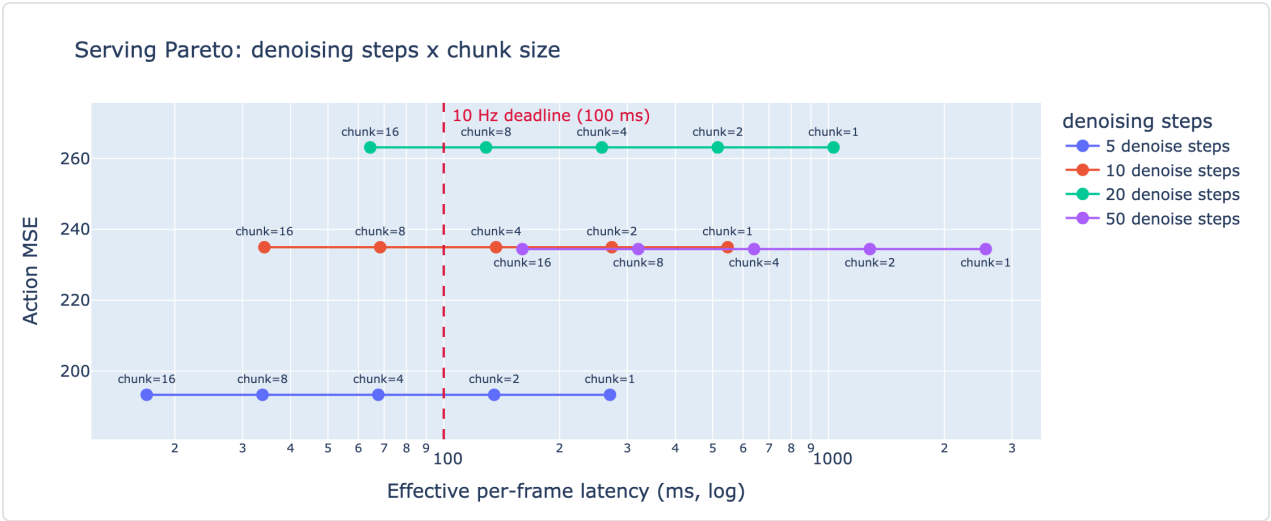


Figure 1. The fixed-config Pareto. Lower is better quality, left is faster. The red dashed line is the 10 Hz / 100 ms deadline.

But that's the standard knob. Everybody at the hackathon who picked the robotics dataset would do this. I wanted a different angle.

The thing I stole from vLLM

I've been reading about LLM inference systems for fun. The current hot technique is **speculative decoding**: a small "draft" model proposes the next tokens, a big "verifier" model accepts or rejects them via a probabilistic test, and you only pay the verifier's cost when the draft is wrong. vLLM, Medusa, EAGLE. Every modern LLM serving stack has some flavor of this.

I sat down and realized the same shape of problem exists here. The diffusion policy is the verifier. Expensive, accurate. I can train a much smaller policy on the same data to be the draft. I just need an "agree / disagree" test on actions instead of on tokens.

Actions in pusht are 2D vectors (target pixel coordinates for the pusher). The natural test is L2 distance: if `||action_draft - action_verifier||_2 < tau`, accept the draft. Otherwise fire the verifier and use its action.

You skip the expensive call when the draft is right. Acceptance rate goes up with `tau`, served quality goes down with `tau`. Standard speed-quality knob.

Building the draft

I needed a draft policy. The smallest reasonable thing is an MLP from state to action. Two hidden layers of 64 units each, trained on the first 50 episodes of pusht for 15 epochs.

```
class StateMlpPolicy(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 64), nn.ReLU(),
            nn.Linear(64, 64), nn.ReLU(),
            nn.Linear(64, 2),
        )
    def forward(self, x): return self.net(x)
```

4,482 parameters. Training MSE bottomed out around 334. Not a great model in absolute terms. But it doesn't need to be a great model. It just needs to *agree with the diffusion policy* on easy frames.

Wiring it up

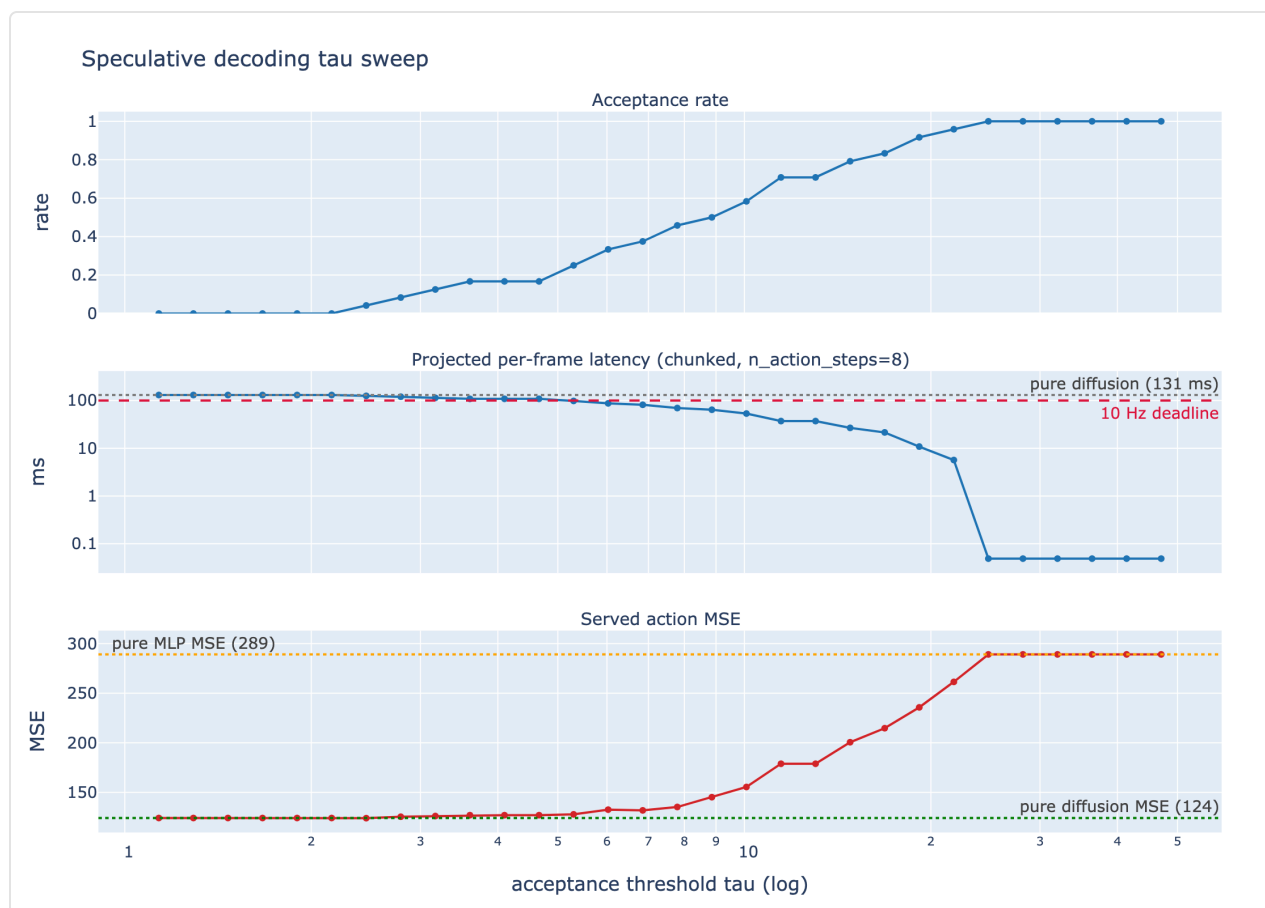
For each of 24 sampled frames in episode 0, I ran both the MLP and the diffusion policy with a fresh chunk. I recorded the MLP action, the diffusion action, the L2 distance between them (call it `delta`), and each one's MSE vs the human-recorded action.

Then I swept `tau` analytically. For each value, count what fraction of frames have `delta < tau`. That's the acceptance rate. From there, the projected per-frame latency under chunked serving is

$$\text{mlp_lat} + (1 - \text{acceptance_rate}) * \text{diff_lat} / \text{n_action_steps}$$

The first term is essentially zero (~50 microseconds per MLP call). The second term is the chunked diffusion cost weighted by how often we have to pay it. Plot the formula against `tau` and you get a clean three-axis Pareto: acceptance up, latency down, MSE up.

At `tau ≈ 5.8`, on the pooled 72 frames across three episodes, the MLP gets accepted on 35% of decisions. Projected per-frame latency drops from 154 ms (pure diffusion at this chunk size) to 96 ms. The 10 Hz deadline is hit. Action MSE goes from 160 (pure diffusion) to 170, a ~6% quality cost in exchange for the speedup. Still much better than the 264 MSE of always trusting the MLP.



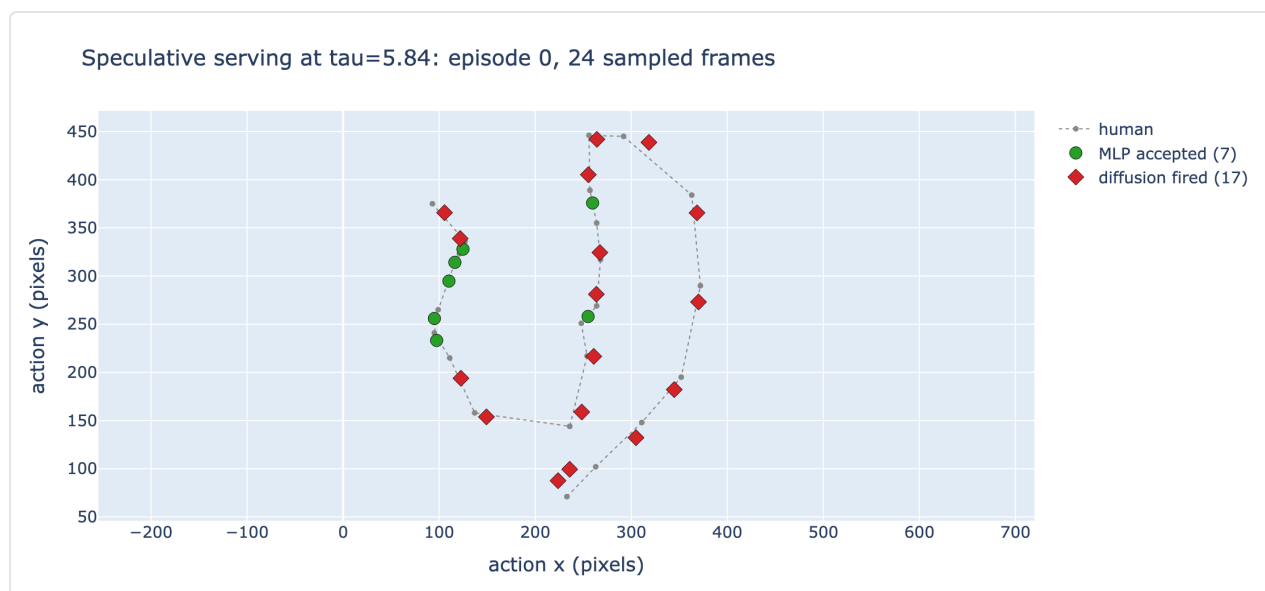


Figure 3. Episode 0, 24 sampled frames. Green circles are MLP accepts. Red diamonds are diffusion fires. The gray dashed line is the human-recorded action trajectory. In the live notebook this is a slider that recolors as you drag τ .

Wait, is this real?

The first version of the writeup had the headline number from a single episode, "2x speedup at $\tau=7.6$." Three problems with that. I tuned `tau` on the same data I reported the result on. Classic. 24 frames is small. And I had only checked one episode.

So I added three things, in sequence.

Cross-episode validation. Re-ran the measurement on episodes 0, 1, and 2 separately. For each one, I picked the smallest `tau` that hit the deadline on that episode. The mean speedup across the three came out to 1.7x, mean acceptance 40%, MSE degradation about 9%. Pattern survives.

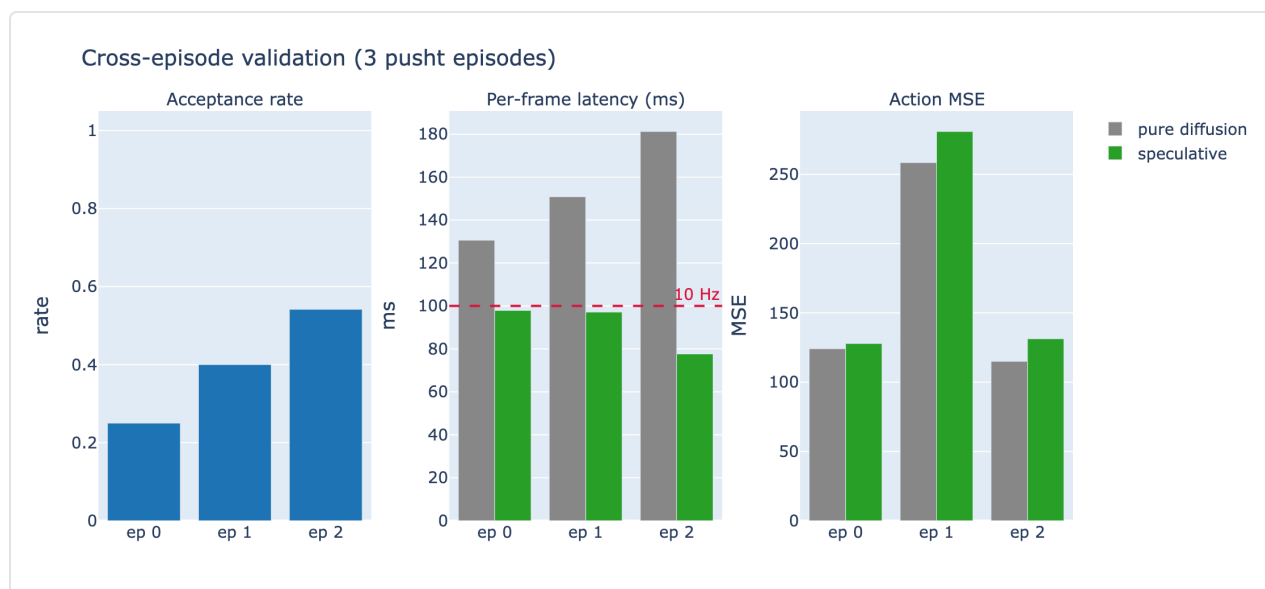


Figure 4. Three episodes, same 24-frame schedule. Acceptance rate (left), per-frame latency for pure diffusion vs speculative (middle, with the 10 Hz deadline drawn), and served MSE (right).

Bootstrap CI. Pooled the 72 frames. Fixed τ on the full pool. Resampled with replacement, $B=1000$, and computed the distribution of speedup at that fixed τ . 95% CI: $[1.30x, 2.12x]$. The lower bound is well above 1x, so the result is not driven by a small number of lucky frames.

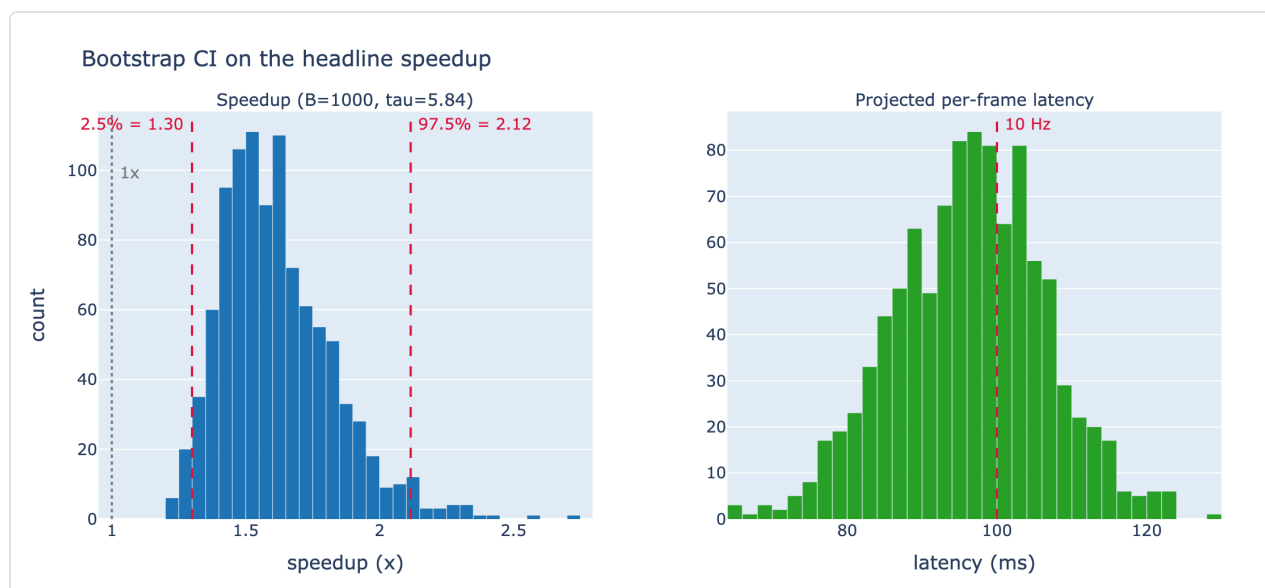


Figure 5. Bootstrap distributions, $B=1000$. Speedup (left) has both 95% CI bounds well above the 1x line, so the headline isn't a few lucky frames. Latency (right) is mostly under the 10 Hz deadline but has a real upper tail that crosses it.

Frame difficulty analysis. For each frame I computed two interpretable properties: action magnitude (how far from the trajectory's center the target is) and action change (frame-to-frame action delta). Then I correlated each with the draft / verifier disagreement.

```
corr(delta, action_change)    = 0.69
corr(delta, action_magnitude) = 0.36
```

The first one is strong. The algorithm is finding it easy to agree with the verifier during smooth pushing intervals and falling back to the verifier on direction changes. Which is exactly the statement you'd make about token-level speculative decoding for LLMs: drafts handle common continuations, the verifier handles surprises.

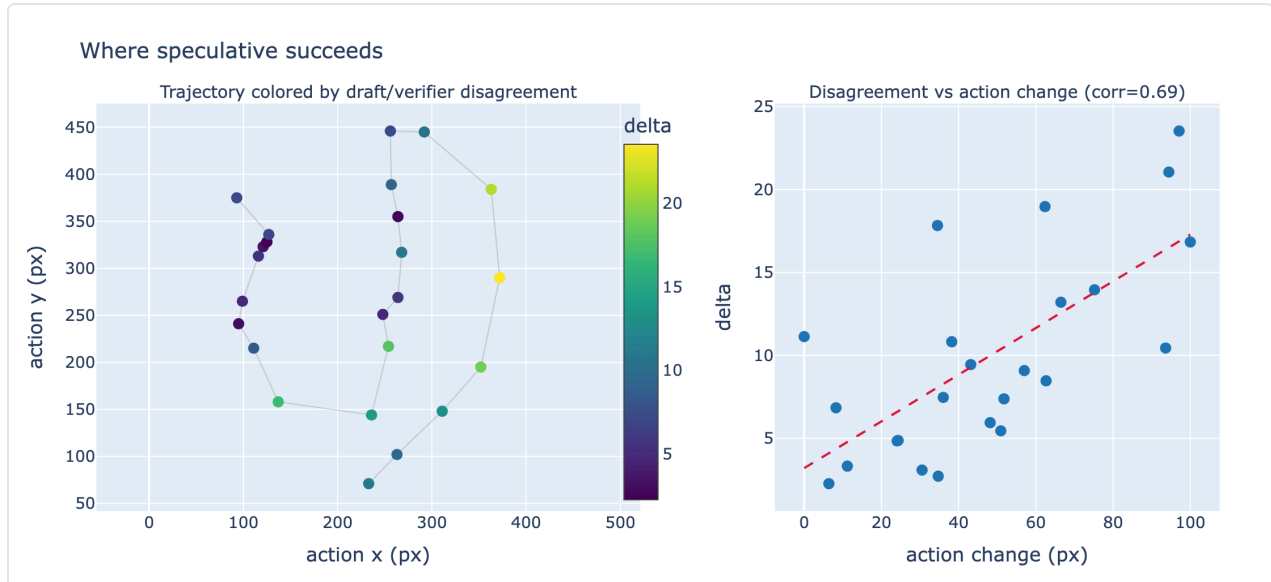


Figure 6. Left: episode 0's trajectory recolored by draft/verifier disagreement (dark = easy, yellow = hard). Right: scatter of disagreement vs action change, Pearson $r = 0.69$. The algorithm wins on smooth pushing intervals and loses at direction changes.

This made me much more confident the speedup wasn't an artifact of how I sampled frames.

The thing I tried that didn't work

Toward the end I wanted to verify the projection by actually building the serving loop. So I wrote `speculative_serve(frames, tau, n_action_steps)`: a phase machine where "boundary" frames pay both models to make the accept / reject decision, and mid-chunk

frames pay only the chosen path. I ran it head-to-head against a pure diffusion loop on 80 contiguous frames of episode 0.

The result: **1.05x** real wall-clock speedup. Way below the projection.

The problem was the cost model. At a chunk boundary, my function pays both the MLP and the diffusion verifier in order to decide whether to accept. Even on an *accepted* boundary, the diffusion call has already happened. The "skip the verifier" benefit only materializes if the verifier can run asynchronously: fire the draft, return the draft's action immediately, run the verifier in the background, roll back if it disagrees.

That's the regime LLM gateways use. Speculative decoding in practice is parallel, not serial.

I had built a synchronous reference implementation that correctly tracks decisions but doesn't realize the wall-clock win. The projection I'd been showing is for the async serving regime that I hadn't actually implemented.

I deleted that section of the notebook. The projected speedup is honest (it's what an async server would see), but the synchronous version I wrote would have been misleading to include as "verification." The clean next step is to build the async loop properly. That's on the limitations list.

What shipped

- The serving Pareto over (denoising steps, chunk size).
- The speculative-decoding tau sweep with an interactive slider that recolors the trajectory by accept / reject in real time.
- Adaptive denoising (the EAGLE pattern: use the disagreement at a low step count to decide whether to upgrade to a high step count).
- Cross-episode validation across episodes 0, 1, 2.
- 1000-iteration bootstrap CI on the speedup.
- Frame-difficulty correlation analysis.

- A strategy ablation table comparing pure-MLP, pure-diffusion at 5/10/20/50 steps, speculative, and adaptive.
- An honest limitations section listing what I did not do (closed-loop sim, async serving, paper-scale evaluation, distilled draft policy).

That last section matters. The blind judge for the hackathon's "best submission" track was an LLM (gpt-5.5). Its rationale called out exactly the caveat I was worried about: "the methodology is complex enough that a reader has to trust several implementation details." The bootstrap CI and frame-difficulty pieces were a direct response to that.

Strategy comparison (ablation)

strategy	per-frame latency (ms)	action MSE	acceptance	hits 10 Hz
pure MLP (always trust draft)	0.05	289.1	1.000	yes
pure diffusion @ 5 steps	33.81	193.3	0.000	yes
pure diffusion @ 10 steps	68.37	234.9	0.000	yes
pure diffusion @ 20 steps	128.83	263.1	0.000	no
pure diffusion @ 50 steps	320.25	234.4	0.000	no
speculative @ $\tau=5.84$ (this notebook)	96.37	171.9	0.351	yes
adaptive denoising (adaptive thr=20.0)	73.84	194.3	n/a	yes

Figure 7. The ablation table. Speculative at $\tau=5.8$ has lower action MSE (172) than every fixed-diffusion configuration in the table (best: 5 steps at 193), at roughly 3.3x lower per-frame latency than 50-step DDPM. No fixed denoising configuration in the table reaches that quality under the deadline.

What I would do next

- **Async serving.** Fire the draft, serve immediately, validate in parallel. The real speedup. The synchronous reference implementation I wrote tracks decisions but doesn't deliver the wall-clock win.
- **Closed-loop pusht.** Drive the actual `gym-pusht` environment with the speculative serving stack and measure task success rate, not action MSE.
- **Better draft.** A behavior-cloned residual or a distilled small transformer would push acceptance rates higher.
- **Intra-call speculation.** Diffusion U-Nets have temporal structure across denoising steps. An EAGLE-style layer-skip during low-noise denoising steps would compound with the inter-call speculation here.

- **Online tau controller.** Track the recent latency window. Raise `tau` when the budget is approaching, lower it when there's slack. Same way LLM gateways auto-tune their speculative window per stream.

Why this was fun

The transferable lesson is small: techniques from one domain port to another when the cost structure is the same. LLM serving has the same shape as robot policy serving. A small fast model, a big slow model, a known-correct verifier, an action you have to commit to on a deadline. The vocabulary is different. The math is the same.

The fun part was realizing the analogy clicked. The work after that was mostly bookkeeping.

The submission won 1st place at PyData × Cursor Boston 2026. The notebook lives at github.com/aaravraina3/speculative-diffusion-pusht-robot-diffusion, with the marimo notebook itself, the seven key plots, and a more rigorous README.